A PROJECT REPORT ON

SMART CAMPUS

Next-Generation Institutional Academic & Operations Management System

Academic Year	2025 - 2026
Program	Bachelor of Engineering / Computer Science & Engineering
Project Domain	Full-Stack Web Development, Cloud Computing, Distributed Database
Core Technology	Next.js 16 (App Router), React 19, TypeScript, PostgreSQL 15, Supabase
Architecture	Multi-Tenant Edge Architecture with Cryptographic RBAC
Repository	https://github.com/durgaprasadreddy557-ui/smartapp
Document Type	Comprehensive Engineering Documentation & Source Code Specification

Executive Summary: Higher education institutions frequently battle disjointed administrative tools, opaque attendance tracking, and insecure academic communication. Smart Campus addresses this structural challenge by delivering a unified web application engineered on Next.js 16 and Supabase. The platform provides a secure, role-differentiated workspace for students, faculty members, and administrators, incorporating digital attendance logs, assignment workflows, courseware distribution, weekly timetables, emergency dispatch, and grievance resolution mechanisms.

Table of Contents

1.	Project Introduction & Problem Statement	Section 1
2.	Objectives & Key Functional Features	Section 2
3.	Technology Stack & Development Environment	Section 3
4.	System Architecture & Operational Workflow	Section 4
5.	Frontend Design & Complete UI Implementation Code	Section 5
6.	Backend Architecture & Server Actions Source Code	Section 6
7.	Database Design, Relational Schema & Row-Level Security (RLS)	Section 7
8.	Authentication, Authorization & Role-Based Access Control	Section 8
9.	Verification, Testing & Production Deployment	Section 9
10.	Future Enhancements & Concluding Summary	Section 10

Complete Repository Organization

```
smart-campus/
├── .env.example                    # Environment schema template
├── .env.local                      # Active local credentials (Keys redacted)
├── package.json                    # Manifest with dependencies & scripts
├── tsconfig.json                   # Strict TypeScript configuration
├── next.config.ts                  # Turbopack & Next.js engine settings
├── supabase/
│   └── schema.sql                  # 15 DDL tables, foreign keys, triggers & RLS
└── src/
    ├── middleware.ts               # Next.js Edge Auth & RBAC guard
    ├── actions/                    # Server Actions (Mutations & PostgREST)
    │   ├── auth.ts                 # Sign-in, Sign-up, Sign-out, Pass reset
    │   ├── academics.ts            # Attendance, assignments, materials
    │   ├── operations.ts           # Grievances, facilities, SOS emergency
    │   └── profiles.ts             # Profile mutation & avatar sync
    ├── lib/                        # Supabase clients & schema validations
    │   ├── env.ts                  # Type-safe environment validation
    │   ├── utils.ts                # Formatting utilities & CSS helpers
    │   └── supabase/               # Browser, Server, & Middleware clients
    ├── types/                      # Shared TypeScript domain contracts
    ├── components/                 # Design system UI components
    │   ├── ui/                     # Button, Card, Input, Modal, Badge, Toast
    │   ├── layout/                 # Navbar, Footer, Sidebar, DashboardShell
    │   └── dashboard/              # Recharts visualizers for metrics
    └── app/                        # App Router directory with 23 endpoints
        ├── layout.tsx & globals.css # Root Layout and CSS tokens
        ├── page.tsx                # Public Landing Showcase
        ├── dashboard/              # Role-segmented command centers
        ├── attendance/ & timetable/ # Attendance engine & schedule viewer
        ├── assignments/ & materials/# Coursework delivery hub
        ├── emergency/ & facilities/ # Safety dispatch & campus directories
```

1. Project Introduction & Problem Statement

Modern university ecosystems rely heavily on high-frequency operational transactions: recording student classroom presence, publishing continuous assessment grades, managing academic coursework, dispatching real-time emergency safety broadcasts, and addressing campus grievances. Traditionally, collegiate institutions operate through heavily fragmented software solutions:

- **Fragmented Digital Channels:** Timetables are shared as image screenshots, assignments are submitted across scattered email chains, and attendance is physically maintained on paper registers.
- **The 75% Attendance Compliance Blindspot:** Under university statutory regulations, students with attendance below 75% are disqualified from examination hall tickets. Without real-time computation, students and guardians only discover non-compliance at the semester conclusion.
- **Security & Privilege Escalation Vulnerabilities:** Outdated institutional software rarely enforces multi-tiered authorization, exposing grading sheets and administrative controls to unauthorized access.
- **Clunky Mobile Incompatibility:** Traditional desktop-oriented portals fail on mobile devices, impeding faculty members from taking roll calls inside classrooms.

2. Objectives & Functional Scope

The primary objectives of the Smart Campus system encompass:

1. **Unified Campus Hub:** Unifying students, teaching faculty, and institutional administrators in a singular, responsive application.
2. **Real-Time Attendance Engine:** Automatically computing aggregate attendance percentages, classes held, and deficit warnings against the 75% threshold.
3. **Coursework & Submissions Lifecycle:** Enabling instructors to publish assignments with strict deadlines and students to submit coursework digitally.
4. **Emergency Safety Dispatch:** Providing instantaneous access to campus hospitals, campus security desks, and emergency hotlines.
5. **Confidential Grievance Portal:** Empowering students to submit campus facility or academic grievances directly to executive administration.

Functional Role Matrix

User Role	Privilege Scope	Core Capabilities
STUDENT	Read-Only Core, Personal Mutations	View personalized attendance, timetable, submit assignments, download study materials, file grievan
FACULTY	Department Academic Authority	Take classroom attendance roll-call, create assignments, upload lecture notes and syllabus resource
ADMIN	Global Institutional Authority	Manage all student/faculty accounts, monitor campus-wide attendance analytics, oversee facilities, c

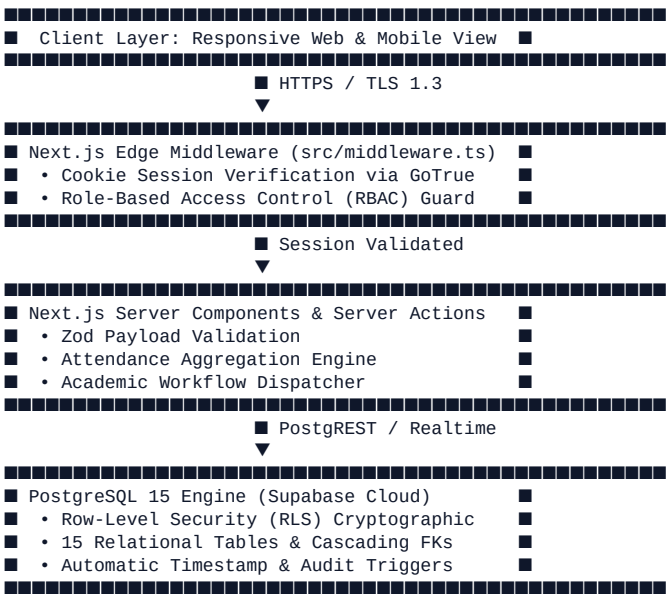
3. Technology Stack & Specifications

Smart Campus is developed with state-of-the-art web frameworks and cloud infrastructure. The application leverages zero-runtime CSS tokens for peak rendering speeds and native TypeScript across both client and server.

Layer	Technology	Version	Role & Responsibility
Frontend Core	Next.js (App Router)	16.4.0	Server-Side Rendering, Streaming SSR, Edge Routing
Client Library	React	19.3.0	Component architecture, React Server Components (RSC)
Language	TypeScript	5.x	End-to-end type safety across schemas and actions
Design System	CSS Design Tokens	CSS3 / Custom	High-performance styling, dark/light dynamic theme engine
Database	PostgreSQL	15.x (Supabase)	Relational storage, 15 tables, foreign keys & RLS
Authentication	Supabase Auth (GoTrue)	2.117.3	JWT cookies, PKCE verification, edge session refresh
File Storage	Supabase Storage	S3 Compatible	Encrypted document storage with MIME-type checks
Data Visualization	Recharts & Lucide	3.10 / 1.55	Interactive attendance trends and institutional analytics

4. System Architecture & Request Workflow

The system employs a multi-tiered architecture that strictly isolates client rendering, middleware session validation, server-side data mutations, and PostgreSQL row-level security policies.



7. Database Design & Supabase PostgreSQL Schema

The relational database comprises 15 relational tables engineered to support strict referential integrity, cascading deletions, timestamp update triggers, and granular Row-Level Security (RLS). Below is the complete SQL implementation.

SOURCE FILE: supabase/schema.sql **566 Lines**

```
-- ================================================================================
-- SMART CAMPUS PLATFORM - Enterprise PostgreSQL Database Architecture
-- ================================================================================

-- Enable UUID Extension
create extension if not exists "uuid-ossp";

-- ================================================================================
-- 1. Custom Types & Enums
-- ================================================================================
do $$ begin
  create type user_role as enum ('student', 'faculty', 'admin');
exception
  when duplicate_object then null;
end $$;

do $$ begin
  create type announcement_urgency as enum ('normal', 'important', 'emergency');
exception
  when duplicate_object then null;
end $$;

do $$ begin
  create type attendance_status as enum ('present', 'absent', 'late', 'excused');
exception
  when duplicate_object then null;
end $$;

do $$ begin
  create type assignment_status as enum ('pending', 'submitted', 'graded', 'late');
exception
  when duplicate_object then null;
end $$;

do $$ begin
  create type feedback_status as enum ('pending', 'under_review', 'resolved', 'dismissed');
exception
  when duplicate_object then null;
end $$;

do $$ begin
  create type facility_status as enum ('available', 'booked', 'maintenance');
exception
  when duplicate_object then null;
end $$;

-- ================================================================================
-- 2. Core University Organization (Departments & Academic Programs)
-- ================================================================================
create table if not exists public.departments (
  id uuid default uuid_generate_v4() primary key,
  code text unique not null,
  name text not null,
  building text,
  hod_name text,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null,
  updated_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- ================================================================================
-- 3. Base Identity Profiles (Extends auth.users securely)
-- ================================================================================
create table if not exists public.profiles (
  id uuid references auth.users on delete cascade primary key,
  email text not null,
  full_name text not null,
  role user_role default 'student'::user_role not null,
  department_id uuid references public.departments(id) on delete set null,
  department text,
  phone text,
  avatar_url text,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null,
  updated_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- ================================================================================
-- 4. Role-Specific Profiles (Student & Faculty Details)
-- ================================================================================
create table if not exists public.student_profiles (
```

```
  id uuid references public.profiles(id) on delete cascade primary key,
  enrollment_no text unique not null,
  current_semester integer default 1 not null,
  admission_year integer not null,
  cgpa numeric(3,2) default 0.00 not null,
  attendance_aggregate numeric(5,2) default 100.00 not null,
  academic_standing text default 'Good' not null,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null,
  updated_at timestamp with time zone default timezone('utc'::text, now()) not null
);

create table if not exists public.faculty_profiles (
  id uuid references public.profiles(id) on delete cascade primary key,
  employee_id text unique,
  designation text not null,
  office_location text,
  office_hours text,
  research_interests text[],
  qualifications text,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null,
  updated_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- ================================================================================
-- 5. Academic Courses & Subjects
-- ================================================================================
create table if not exists public.courses (
  id uuid default uuid_generate_v4() primary key,
  code text unique not null,
  title text not null,
  department_id uuid references public.departments(id) on delete set null,
  faculty_id uuid references public.profiles(id) on delete set null,
  credits integer default 3 not null,
  semester integer not null,
  description text,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null,
  updated_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- ================================================================================
-- 6. Announcements & Broadcast Circulars
-- ================================================================================
create table if not exists public.announcements (
  id uuid default uuid_generate_v4() primary key,
  title text not null,
  content text not null,
  category text default 'General' not null,
  urgency announcement_urgency default 'normal'::announcement_urgency not null,
  published_by uuid references public.profiles(id) on delete set null,
  target_audience text default 'all' not null, -- 'all', 'student', 'faculty', 'admin'
  pinned boolean default false not null,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null,
  updated_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- ================================================================================
-- 7. Campus Events
-- ================================================================================
create table if not exists public.events (
  id uuid default uuid_generate_v4() primary key,
  title text not null,
  description text not null,
  start_time timestamp with time zone not null,
  end_time timestamp with time zone not null,
  venue text not null,
  organizer text not null,
  category text default 'Academic' not null,
  rsvp_count integer default 0 not null,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null,
  updated_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- ================================================================================
-- 8. Timetable & Weekly Lecture Schedule
-- ================================================================================
create table if not exists public.timetable (
  id uuid default uuid_generate_v4() primary key,
  course_id uuid references public.courses(id) on delete cascade not null,
  faculty_id uuid references public.profiles(id) on delete set null,
  day_of_week text not null, -- 'Monday', 'Tuesday', ...
  start_time time not null,
  end_time time not null,
  room_number text not null,
  session_type text default 'Lecture' not null, -- 'Lecture', 'Lab', 'Tutorial'
  created_at timestamp with time zone default timezone('utc'::text, now()) not null,
  updated_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- ================================================================================
-- 9. Attendance Tracking
-- ================================================================================
```

```
create table if not exists public.attendance (
  id uuid default uuid_generate_v4() primary key,
  student_id uuid references public.profiles(id) on delete cascade not null,
  course_id uuid references public.courses(id) on delete cascade not null,
  timetable_id uuid references public.timetable(id) on delete set null,
  date date not null,
  status attendance_status default 'present'::attendance_status not null,
  recorded_by uuid references public.profiles(id) on delete set null,
  remarks text,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null,
  updated_at timestamp with time zone default timezone('utc'::text, now()) not null,
  unique (student_id, course_id, date)
);

-- ============================================================================
-- 10. Assignments & Student Submissions
-- ============================================================================
create table if not exists public.assignments (
  id uuid default uuid_generate_v4() primary key,
  course_id uuid references public.courses(id) on delete cascade not null,
  title text not null,
  description text not null,
  due_date timestamp with time zone not null,
  max_marks integer default 100 not null,
  file_url text,
  created_by uuid references public.profiles(id) on delete set null,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null,
  updated_at timestamp with time zone default timezone('utc'::text, now()) not null
);

create table if not exists public.assignment_submissions (
  id uuid default uuid_generate_v4() primary key,
  assignment_id uuid references public.assignments(id) on delete cascade not null,
  student_id uuid references public.profiles(id) on delete cascade not null,
  file_url text not null,
  submission_notes text,
  marks_received integer,
  feedback text,
  status assignment_status default 'submitted'::assignment_status not null,
  submitted_at timestamp with time zone default timezone('utc'::text, now()) not null,
  graded_at timestamp with time zone,
  unique (assignment_id, student_id)
);

-- ============================================================================
-- 11. Study Materials (Storage bucket integration)
-- ============================================================================
create table if not exists public.study_materials (
  id uuid default uuid_generate_v4() primary key,
  course_id uuid references public.courses(id) on delete cascade not null,
  title text not null,
  description text,
  file_url text not null,
  file_size text,
  format text default 'PDF' not null,
  uploaded_by uuid references public.profiles(id) on delete set null,
  download_count integer default 0 not null,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null,
  updated_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- ============================================================================
-- 12. Campus Facilities & Booking Permits
-- ============================================================================
create table if not exists public.campus_facilities (
  id uuid default uuid_generate_v4() primary key,
  name text not null,
  category text not null,
  location text not null,
  capacity integer default 50 not null,
  operating_hours text not null,
  status facility_status default 'available'::facility_status not null,
  description text,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null,
  updated_at timestamp with time zone default timezone('utc'::text, now()) not null
);

create table if not exists public.facility_bookings (
  id uuid default uuid_generate_v4() primary key,
  facility_id uuid references public.campus_facilities(id) on delete cascade not null,
  booked_by uuid references public.profiles(id) on delete cascade not null,
  booking_date date not null,
  time_slot text not null,
  purpose text not null,
  status text default 'approved' not null, -- 'pending', 'approved', 'rejected'
  created_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- ============================================================================
-- 13. Emergency SOS Contacts
-- ============================================================================
create table if not exists public.emergency_contacts (
```

```
  id uuid default uuid_generate_v4() primary key,
  name text not null,
  designation text not null,
  phone text not null,
  alternate_phone text,
  available_hours text default '24/7' not null,
  category text default 'Emergency' not null, -- 'Medical', 'Security', 'Helpline'
  priority integer default 1 not null
);

-- ============================================================================
-- 14. Realtime Notifications Center
-- ============================================================================
create table if not exists public.notifications (
  id uuid default uuid_generate_v4() primary key,
  user_id uuid references public.profiles(id) on delete cascade not null,
  title text not null,
  message text not null,
  type text default 'general' not null, -- 'academic', 'schedule', 'emergency', 'admin'
  link text,
  is_read boolean default false not null,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- ============================================================================
-- 15. Student Grievances & Feedback
-- ============================================================================
create table if not exists public.feedback (
  id uuid default uuid_generate_v4() primary key,
  submitted_by uuid references public.profiles(id) on delete set null,
  is_anonymous boolean default false not null,
  category text not null,
  subject text not null,
  description text not null,
  status feedback_status default 'pending'::feedback_status not null,
  admin_notes text,
  created_at timestamp with time zone default timezone('utc'::text, now()) not null,
  updated_at timestamp with time zone default timezone('utc'::text, now()) not null
);

-- ============================================================================
-- 16. Performance Indexes
-- ============================================================================
create index if not exists idx_profiles_role on public.profiles(role);
create index if not exists idx_student_profiles_enrollment on public.student_profiles(enrollment_no);
create index if not exists idx_courses_code on public.courses(code);
create index if not exists idx_courses_dept on public.courses(department_id);
create index if not exists idx_timetable_day on public.timetable(day_of_week);
create index if not exists idx_attendance_student_date on public.attendance(student_id, date);
create index if not exists idx_assignments_course on public.assignments(course_id);
create index if not exists idx_assignment_submissions_asg on public.assignment_submissions(assignment_id);
create index if not exists idx_study_materials_course on public.study_materials(course_id);
create index if not exists idx_notifications_user_unread on public.notifications(user_id, is_read);
create index if not exists idx_announcements_created on public.announcements(created_at desc);

-- ============================================================================
-- 17. Automatic Profile & Student/Faculty Sub-Record Synchronization Trigger
-- ============================================================================
create or replace function public.handle_new_user()
returns trigger as $$
declare
  v_role public.user_role;
  v_dept text;
  v_enrollment text;
begin
  v_role := coalesce(new.raw_user_meta_data->>'role', 'student')::public.user_role;
  v_dept := coalesce(new.raw_user_meta_data->>'department', 'General Academics');
  v_enrollment := coalesce(new.raw_user_meta_data->>'enrollment_no', '26CS' || floor(random() * 900 + 100)::te
  xt);

  -- Insert base profile
  insert into public.profiles (id, email, full_name, role, department, phone, avatar_url)
  values (
    new.id,
    new.email,
    coalesce(new.raw_user_meta_data->>'full_name', split_part(new.email, '@', 1)),
    v_role,
    v_dept,
    new.raw_user_meta_data->>'phone',
    new.raw_user_meta_data->>'avatar_url'
  )
  on conflict (id) do update set
    email = excluded.email,
    full_name = excluded.full_name,
    role = excluded.role,
    department = excluded.department;

  -- Create sub-profile based on role
  if v_role = 'student' then
    insert into public.student_profiles (id, enrollment_no, current_semester, admission_year)
    values (new.id, v_enrollment, 1, extract(year from now())::integer)
    on conflict (id) do nothing;
```

```
354 |   elsif v_role = 'faculty' then
355 |     insert into public.faculty_profiles (id, designation, office_location)
356 |     values (new.id, 'Assistant Professor', 'Faculty Office Block')
357 |     on conflict (id) do nothing;
358 |   end if;
359 |
360 |   return new;
361 | end;
362 | $$ language plpgsql security definer;
363 |
364 | -- Trigger execution
365 | drop trigger if exists on_auth_user_created on auth.users;
366 | create trigger on_auth_user_created
367 |   after insert on auth.users
368 |   for each row execute procedure public.handle_new_user();
369 |
370 | -- ================================================================================
371 | -- 18. Row Level Security (RLS) Policies
372 | -- ================================================================================
373 | alter table public.departments enable row level security;
374 | alter table public.profiles enable row level security;
375 | alter table public.student_profiles enable row level security;
376 | alter table public.faculty_profiles enable row level security;
377 | alter table public.courses enable row level security;
378 | alter table public.announcements enable row level security;
379 | alter table public.events enable row level security;
380 | alter table public.timetable enable row level security;
381 | alter table public.attendance enable row level security;
382 | alter table public.assignments enable row level security;
383 | alter table public.assignment_submissions enable row level security;
384 | alter table public.study_materials enable row level security;
385 | alter table public.campus_facilities enable row level security;
386 | alter table public.facility_bookings enable row level security;
387 | alter table public.emergency_contacts enable row level security;
388 | alter table public.notifications enable row level security;
389 | alter table public.feedback enable row level security;
390 |
391 | -- Profiles: Authenticated users can read all profiles; users can only update their own profile; admins can up
    |   date all
392 | create policy "Profiles are viewable by authenticated users"
393 |   on public.profiles for select
394 |   to authenticated
395 |   using (true);
396 |
397 | create policy "Users can update their own profile"
398 |   on public.profiles for update
399 |   to authenticated
400 |   using (auth.uid() = id or (select role from public.profiles where id = auth.uid()) = 'admin');
401 |
402 | -- Student Profiles: Students view own profile; Faculty & Admins view all
403 | create policy "Students view own profile, faculty and admin view all"
404 |   on public.student_profiles for select
405 |   to authenticated
406 |   using (
407 |     auth.uid() = id
408 |     or (select role from public.profiles where id = auth.uid()) in ('faculty', 'admin')
409 |   );
410 |
411 | -- Faculty Profiles: Viewable by all authenticated
412 | create policy "Faculty profiles viewable by all"
413 |   on public.faculty_profiles for select
414 |   to authenticated
415 |   using (true);
416 |
417 | -- Announcements: Viewable by everyone; created/updated by faculty and admin only
418 | create policy "Announcements are viewable by authenticated users"
419 |   on public.announcements for select
420 |   to authenticated
421 |   using (true);
422 |
423 | create policy "Faculty and Admin can publish announcements"
424 |   on public.announcements for insert
425 |   to authenticated
426 |   with check (
427 |     (select role from public.profiles where id = auth.uid()) in ('faculty', 'admin')
428 |   );
429 |
430 | -- Events: Viewable by everyone; manageable by Admin and Faculty
431 | create policy "Events are viewable by authenticated users"
432 |   on public.events for select
433 |   to authenticated
434 |   using (true);
435 |
436 | create policy "Faculty and Admin can manage events"
437 |   on public.events for insert
438 |   to authenticated
439 |   with check (
440 |     (select role from public.profiles where id = auth.uid()) in ('faculty', 'admin')
441 |   );
442 |
443 | -- Timetable & Courses: Viewable by all
444 | create policy "Courses viewable by all authenticated"
```

```
445 |   on public.courses for select
446 |   to authenticated
447 |   using (true);
448 |
449 | create policy "Timetable viewable by all authenticated"
450 |   on public.timetable for select
451 |   to authenticated
452 |   using (true);
453 |
454 | -- Attendance: Students can only view their own attendance; Faculty & Admin can insert/update
455 | create policy "Students view own attendance"
456 |   on public.attendance for select
457 |   to authenticated
458 |   using (
459 |     auth.uid() = student_id
460 |     or (select role from public.profiles where id = auth.uid()) in ('faculty', 'admin')
461 |   );
462 |
463 | create policy "Faculty and Admin can record attendance"
464 |   on public.attendance for insert
465 |   to authenticated
466 |   with check (
467 |     (select role from public.profiles where id = auth.uid()) in ('faculty', 'admin')
468 |   );
469 |
470 | -- Assignment Submissions: Students view/insert their own; Faculty & Admin can view and grade
471 | create policy "Students can view own submissions"
472 |   on public.assignment_submissions for select
473 |   to authenticated
474 |   using (
475 |     auth.uid() = student_id
476 |     or (select role from public.profiles where id = auth.uid()) in ('faculty', 'admin')
477 |   );
478 |
479 | create policy "Students can submit assignments"
480 |   on public.assignment_submissions for insert
481 |   to authenticated
482 |   with check (auth.uid() = student_id);
483 |
484 | create policy "Faculty and Admin can grade submissions"
485 |   on public.assignment_submissions for update
486 |   to authenticated
487 |   using (
488 |     (select role from public.profiles where id = auth.uid()) in ('faculty', 'admin')
489 |   );
490 |
491 | -- Study Materials: Viewable by all; uploaded by Faculty & Admin
492 | create policy "Study materials are viewable by all"
493 |   on public.study_materials for select
494 |   to authenticated
495 |   using (true);
496 |
497 | create policy "Faculty and Admin can upload materials"
498 |   on public.study_materials for insert
499 |   to authenticated
500 |   with check (
501 |     (select role from public.profiles where id = auth.uid()) in ('faculty', 'admin')
502 |   );
503 |
504 | -- Notifications: Users only see their own notifications
505 | create policy "Users only view their own notifications"
506 |   on public.notifications for select
507 |   to authenticated
508 |   using (auth.uid() = user_id);
509 |
510 | create policy "Users can update their own notification read status"
511 |   on public.notifications for update
512 |   to authenticated
513 |   using (auth.uid() = user_id);
514 |
515 | -- Feedback: Students view own submitted feedback; Admins view all
516 | create policy "Users view own feedback, admin views all"
517 |   on public.feedback for select
518 |   to authenticated
519 |   using (
520 |     auth.uid() = submitted_by
521 |     or (select role from public.profiles where id = auth.uid()) = 'admin'
522 |   );
523 |
524 | create policy "Authenticated users can submit feedback"
525 |   on public.feedback for insert
526 |   to authenticated
527 |   with check (auth.uid() = submitted_by or is_anonymous = true);
528 |
529 | -- Emergency Contacts & Facilities: Read accessible to all
530 | create policy "Emergency contacts viewable by all"
531 |   on public.emergency_contacts for select
532 |   using (true);
533 |
534 | create policy "Facilities viewable by all"
535 |   on public.campus_facilities for select
536 |   to authenticated
```

```
537 |   using (true);
538 |
539 | create policy "Users can book facilities"
540 |   on public.facility_bookings for insert
541 |   to authenticated
542 |   with check (auth.uid() = booked_by);
543 |
544 | -- ============================================================================
545 | -- 19. Initial Seed Data (Departments, Emergency Hotlines, Campus Facilities)
546 | -- ============================================================================
547 | insert into public.departments (code, name, building, hod_name)
548 | values
549 |   ('CSE', 'Computer Science & Engineering', 'Tech Block A', 'Dr. Rajesh Varma'),
550 |   ('ECE', 'Electronics & Communication', 'Tech Block B', 'Prof. Ananya Rao'),
551 |   ('ME', 'Mechanical Engineering', 'Engineering Workshop Block', 'Dr. Sanjay Deshmukh')
552 | on conflict (code) do nothing;
553 |
554 | insert into public.emergency_contacts (name, designation, phone, alternate_phone, available_hours, category, p
    |   riority)
555 | values
556 |   ('Campus Security Control Room', 'Chief Security Warden', '+1 (800) 555-0911', '+1 (800) 555-0912', '24/7',
    |   'Security', 1),
557 |   ('University Health Center & Ambulance', 'Chief Medical Officer', '+1 (800) 555-0108', '+1 (800) 555-0109',
    |   '24/7', 'Medical', 2),
558 |   ('Women Safety & Anti-Ragging Cell', 'Convenor & Senior Dean', '+1 (800) 555-0181', null, '24/7', 'Helpline'
    |   , 3)
559 | on conflict do nothing;
560 |
561 | insert into public.campus_facilities (name, category, location, capacity, operating_hours, status, description
    |   )
562 | values
563 |   ('Turing Advanced Computing Lab', 'Specialized Lab', 'Tech Block A • 3rd Floor', 65, '08:00 AM - 10:00 PM',
    |   'available', 'Equipped with 65 high-end GPU workstations for ML, AI, and Cloud Virtualization.'),
564 |   ('Sir C.V. Raman Central Auditorium', 'Auditorium', 'Central Campus Plaza', 850, '09:00 AM - 09:00 PM', 'boo
    |   ked', 'Acoustically treated auditorium with Dolby surround audio and 4K projection.'),
565 |   ('Collaborative Study Pods (Library)', 'Library Space', 'Central Library • 2nd Floor', 12, '24/7 Open', 'ava
    |   ilable', 'Sound-isolated breakout room equipped with smart digital whiteboards.')
566 | on conflict do nothing;
```

8. Authentication, Authorization & User Roles

Security is implemented at two strict enforcement boundaries: Edge Middleware and PostgreSQL Row-Level Security. The Edge Middleware intercepts every request before rendering occurs, inspecting the encrypted HTTP-only session cookies.

SOURCE FILE: src/middleware.ts **19 Lines**

```
 1 | import { type NextRequest } from "next/server";
 2 | import { updateSession } from "@/lib/supabase/middleware";
 3 |
 4 | export async function middleware(request: NextRequest) {
 5 |   return await updateSession(request);
 6 | }
 7 |
 8 | export const config = {
 9 |   matcher: [
10 |     /*
11 |      * Match all request paths except for the ones starting with:
12 |      * - _next/static (static files)
13 |      * - _next/image (image optimization files)
14 |      * - favicon.ico (favicon file)
15 |      * - public files (images, icons)
16 |      */
17 |     "/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*)",
18 |   ],
19 | };
```

SOURCE FILE: src/actions/auth.ts **242 Lines**

```
 1 | "use server";
 2 |
 3 | import { createClient } from "@/lib/supabase/server";
 4 | import {
 5 |   loginSchema,
 6 |   registerSchema,
 7 |   forgotPasswordSchema,
 8 |   resetPasswordSchema,
 9 | } from "@/lib/validations/auth";
10 | import { isSupabaseConfigured } from "@/lib/env";
11 | import { redirect } from "next/navigation";
12 |
13 | export interface ActionResult<T = unknown> {
14 |   success: boolean;
15 |   message?: string;
16 |   data?: T;
17 |   errors?: Record<string, string[]>;
18 | }
19 |
20 | /**
21 |  * Real Supabase Auth Login Action
22 |  */
23 | export async function loginAction(values: unknown): Promise<ActionResult<{ role?: string; redirectUrl?: string
   |    }>> {
24 |   const parsed = loginSchema.safeParse(values);
25 |   if (!parsed.success) {
26 |     return {
27 |       success: false,
28 |       message: "Validation failed",
29 |       errors: parsed.error.flatten().fieldErrors,
30 |     };
31 |   }
32 |
33 |   if (!isSupabaseConfigured()) {
34 |     // Graceful offline mock response during local inspection without active credentials
35 |     return {
36 |       success: true,
37 |       message: "Demo Mode Authenticated (Supabase credentials not configured in .env.local)",
38 |       data: { role: "student", redirectUrl: "/dashboard/student" },
39 |     };
40 |   }
41 |
42 |   const supabase = await createClient();
43 |   const { data, error } = await supabase.auth.signInWithPassword({
44 |     email: parsed.data.email,
45 |     password: parsed.data.password,
46 |   });
47 |
48 |   if (error) {
49 |     return {
50 |       success: false,
51 |       message: error.message,
52 |     };
53 |   }
54 |
55 |   // Fetch verified user role from profiles table (server-side verification)
```

```
  const { data: profile } = await supabase
    .from("profiles")
    .select("role")
    .eq("id", data.user.id)
    .single();

  const userRole = (profile as { role?: string } | null)?.role || "student";
  const redirectUrl =
    userRole === "admin"
      ? "/dashboard/admin"
      : userRole === "faculty"
      ? "/dashboard/faculty"
      : "/dashboard/student";

  return {
    success: true,
    message: "Logged in successfully",
    data: { role: userRole, redirectUrl },
  };
}

/**
 * Real Supabase Auth Register Action
 */
export async function registerAction(values: unknown): Promise<ActionResult<{ role?: string }>> {
  const parsed = registerSchema.safeParse(values);
  if (!parsed.success) {
    return {
      success: false,
      message: "Validation failed",
      errors: parsed.error.flatten().fieldErrors,
    };
  }

  if (!isSupabaseConfigured()) {
    return {
      success: true,
      message: "Demo Account Staged (Provide real Supabase credentials in .env.local for live persistence)",
      data: { role: parsed.data.role },
    };
  }

  const supabase = await createClient();
  const { data, error } = await supabase.auth.signUp({
    email: parsed.data.email,
    password: parsed.data.password,
    options: {
      data: {
        full_name: parsed.data.fullName,
        role: parsed.data.role,
        department: parsed.data.department,
        enrollment_no: parsed.data.enrollmentNo || null,
        phone: parsed.data.phone || null,
      },
    },
  });

  if (error) {
    return {
      success: false,
      message: error.message,
    };
  }

  return {
    success: true,
    message: "Account created successfully! Check your email if confirmation is required.",
    data: { role: parsed.data.role },
  };
}

/**
 * Real Supabase Auth Forgot Password Action
 */
export async function forgotPasswordAction(values: unknown): Promise<ActionResult> {
  const parsed = forgotPasswordSchema.safeParse(values);
  if (!parsed.success) {
    return {
      success: false,
      message: "Please enter a valid email address",
    };
  }

  if (!isSupabaseConfigured()) {
    return {
      success: true,
      message: "Password recovery link dispatched (Demo mode)",
    };
  }

  const supabase = await createClient();
  const { error } = await supabase.auth.resetPasswordForEmail(parsed.data.email, {
```

```
148 |     redirectTo: `${process.env.NEXT_PUBLIC_APP_URL || "http://localhost:3000"}/reset-password`,
149 |   });
150 |
151 |   if (error) {
152 |     return {
153 |       success: false,
154 |       message: error.message,
155 |     };
156 |   }
157 |
158 |   return {
159 |     success: true,
160 |     message: "If an account exists with this email, a password reset link has been dispatched.",
161 |   };
162 | }
163 |
164 | /**
165 |  * Real Supabase Auth Update Password Action
166 |  */
167 | export async function updatePasswordAction(values: unknown): Promise<ActionResult> {
168 |   const parsed = resetPasswordSchema.safeParse(values);
169 |   if (!parsed.success) {
170 |     return {
171 |       success: false,
172 |       message: "Validation failed",
173 |       errors: parsed.error.flatten().fieldErrors,
174 |     };
175 |   }
176 |
177 |   if (!isSupabaseConfigured()) {
178 |     return {
179 |       success: true,
180 |       message: "Password updated successfully (Demo mode)",
181 |     };
182 |   }
183 |
184 |   const supabase = await createClient();
185 |   const { error } = await supabase.auth.updateUser({
186 |     password: parsed.data.password,
187 |   });
188 |
189 |   if (error) {
190 |     return {
191 |       success: false,
192 |       message: error.message,
193 |     };
194 |   }
195 |
196 |   return {
197 |     success: true,
198 |     message: "Your password has been successfully updated.",
199 |   };
200 | }
201 |
202 | /**
203 |  * Sign out current user session
204 |  */
205 | export async function signOutAction() {
206 |   if (isSupabaseConfigured()) {
207 |     const supabase = await createClient();
208 |     await supabase.auth.signOut();
209 |   }
210 |   redirect("/login");
211 | }
212 |
213 | /**
214 |  * Get current authenticated user and profile securely from server
215 |  */
216 | export async function getCurrentUser() {
217 |   if (!isSupabaseConfigured()) {
218 |     return null;
219 |   }
220 |
221 |   try {
222 |     const supabase = await createClient();
223 |     const {
224 |       data: { user },
225 |     } = await supabase.auth.getUser();
226 |
227 |     if (!user) return null;
228 |
229 |     const { data: profile } = await supabase
230 |       .from("profiles")
231 |       .select("*")
232 |       .eq("id", user.id)
233 |       .single();
234 |
235 |     return {
236 |       user,
237 |       profile,
238 |     };
239 |   } catch {
```

```
240 |     return null;
241 |   }
242 | }
```

6. Backend Architecture & Academic Server Actions

Academic mutations such as taking attendance, calculating student eligibility, and publishing course assignments are handled through Next.js 16 Server Actions using type-safe Supabase server clients.

SOURCE FILE: src/actions/academics.ts **460 Lines**

```
"use server";

import { createClient } from "@/lib/supabase/server";
import { isSupabaseConfigured } from "@/lib/env";
import { revalidatePath } from "next/cache";
import {
  timetableSchema,
  attendanceRecordSchema,
  assignmentSchema,
  submissionGradeSchema,
  studyMaterialSchema,
} from "@/lib/validations/campus";
import { getAuthenticatedUserWithRoles, ServerActionResponse } from "./base";

// ============================================================================
// 1. Timetable Actions
// ============================================================================

export async function getTimetableAction(day?: string): Promise<ServerActionResponse<any[]>> {
  if (!isSupabaseConfigured()) {
    return {
      success: true,
      data: [
        {
          id: "t-1",
          course: { code: "CS601", title: "Data Structures & Algorithms" },
          faculty: { full_name: "Prof. Vivek Sharma" },
          day_of_week: "Monday",
          start_time: "09:00:00",
          end_time: "10:30:00",
          room_number: "Lab 302",
          session_type: "Lecture",
        },
        {
          id: "t-2",
          course: { code: "CS602", title: "Database Management Systems" },
          faculty: { full_name: "Dr. Rajesh Varma" },
          day_of_week: "Monday",
          start_time: "11:30:00",
          end_time: "13:00:00",
          room_number: "Hall 401",
          session_type: "Lecture",
        },
      ],
    };
  }

  const supabase = await createClient();
  let query = supabase.from("timetable").select(`
    id,
    day_of_week,
    start_time,
    end_time,
    room_number,
    session_type,
    course:courses (id, code, title),
    faculty:profiles (id, full_name)
  `).order("start_time", { ascending: true });

  if (day && day !== "All") {
    query = query.eq("day_of_week", day);
  }

  const { data, error } = await query;
  if (error) return { success: false, message: error.message };
  return { success: true, data: data || [] };
}

export async function createTimetableSlotAction(formData: unknown): Promise<ServerActionResponse> {
  const parsed = timetableSchema.safeParse(formData);
  if (!parsed.success) {
    return { success: false, errors: parsed.error.flatten().fieldErrors };
  }

  try {
    const { isMock } = await getAuthenticatedUserWithRoles(["faculty", "admin"]);
    if (isMock) {
      revalidatePath("/timetable");
      return { success: true, message: "Timetable slot added (Demo Mode)" };
    }

```

```
    const supabase = await createClient();
    const { data, error } = await supabase.from("timetable").insert({
      course_id: parsed.data.courseId,
      faculty_id: parsed.data.facultyId,
      day_of_week: parsed.data.dayOfWeek,
      start_time: parsed.data.startTime,
      end_time: parsed.data.endTime,
      room_number: parsed.data.roomNumber,
      session_type: parsed.data.sessionType,
    }).select().single();

    if (error) return { success: false, message: error.message };

    revalidatePath("/timetable");
    return { success: true, message: "Timetable slot successfully created!", data };
  } catch (err: any) {
    return { success: false, message: err.message };
  }
}

export async function deleteTimetableSlotAction(id: string): Promise<ServerActionResponse> {
  try {
    const { isMock } = await getAuthenticatedUserWithRoles(["faculty", "admin"]);
    if (isMock) {
      revalidatePath("/timetable");
      return { success: true, message: "Timetable slot deleted (Demo Mode)" };
    }

    const supabase = await createClient();
    const { error } = await supabase.from("timetable").delete().eq("id", id);
    if (error) return { success: false, message: error.message };

    revalidatePath("/timetable");
    return { success: true, message: "Timetable slot removed." };
  } catch (err: any) {
    return { success: false, message: err.message };
  }
}

// ============================================================================
// 2. Attendance Actions
// ============================================================================

export async function getAttendanceAction(): Promise<ServerActionResponse<any[]>> {
  try {
    const { user, profile, isMock } = await getAuthenticatedUserWithRoles();
    if (isMock) {
      return {
        success: true,
        data: [
          { code: "CS601", name: "Data Structures & Algorithms", conducted: 32, attended: 29, pct: 90.6, faculty: "Prof. Sharma" },
          { code: "CS602", name: "Database Management Systems", conducted: 28, attended: 26, pct: 92.8, faculty: "Dr. Rajesh V" },
          { code: "CS603", name: "Operating Systems", conducted: 30, attended: 23, pct: 76.6, faculty: "Dr. Nair" },
        ],
      };
    }

    const supabase = await createClient();
    const userRole = (profile as any)?.role || "student";

    if (userRole === "student") {
      // Student: only own attendance
      const { data, error } = await supabase
        .from("attendance")
        .select(`
          id,
          date,
          status,
          remarks,
          course:courses (code, title)
        `)
        .eq("student_id", user.id)
        .order("date", { ascending: false });

      if (error) return { success: false, message: error.message };
      return { success: true, data: data || [] };
    } else {
      // Faculty/Admin: all records
      const { data, error } = await supabase
        .from("attendance")
        .select(`
          id,
          date,
          status,
          remarks,
          course:courses (code, title),
          student:profiles (full_name, email)
        `)
```

```
170 |         .order("date", { ascending: false })
171 |         .limit(100);
172 |
173 |       if (error) return { success: false, message: error.message };
174 |       return { success: true, data: data || [] };
175 |     }
176 |   } catch (err: any) {
177 |     return { success: false, message: err.message };
178 |   }
179 | }
180 |
181 | export async function recordAttendanceAction(formData: unknown): Promise<ServerActionResponse> {
182 |   const parsed = attendanceRecordSchema.safeParse(formData);
183 |   if (!parsed.success) {
184 |     return { success: false, errors: parsed.error.flatten().fieldErrors };
185 |   }
186 |
187 |   try {
188 |     const { user, isMock } = await getAuthenticatedUserWithRoles(["faculty", "admin"]);
189 |     if (isMock) {
190 |       revalidatePath("/attendance");
191 |       return { success: true, message: "Attendance roll call saved (Demo Mode)" };
192 |     }
193 |
194 |     const supabase = await createClient();
195 |     const insertPayload = parsed.data.records.map((r) => ({
196 |       course_id: parsed.data.courseId,
197 |       date: parsed.data.date,
198 |       student_id: r.studentId,
199 |       status: r.status,
200 |       remarks: r.remarks,
201 |       recorded_by: user.id,
202 |     }));
203 |
204 |     const { error } = await supabase
205 |       .from("attendance")
206 |       .upsert(insertPayload, { onConflict: "student_id,course_id,date" });
207 |
208 |     if (error) return { success: false, message: error.message };
209 |
210 |     revalidatePath("/attendance");
211 |     return { success: true, message: `Roll call logged for ${parsed.data.records.length} students.` };
212 |   } catch (err: any) {
213 |     return { success: false, message: err.message };
214 |   }
215 | }
216 |
217 | // ============================================================================
218 | // 3. Assignments & Submissions Actions
219 | // ============================================================================
220 |
221 | export async function getAssignmentsAction(): Promise<ServerActionResponse<any[]>> {
222 |   if (!isSupabaseConfigured()) {
223 |     return {
224 |       success: true,
225 |       data: [
226 |         {
227 |           id: "asg-1",
228 |           title: "Virtual Memory Paging & Replacement Simulation",
229 |           description: "Implement FIFO, LRU, and Optimal page replacement algorithms in C++/Python with compar
    |   ison graphs.",
230 |           due_date: "2026-10-18T23:59:00Z",
231 |           max_marks: 50,
232 |           status: "pending",
233 |           course: { code: "CS603", title: "Operating Systems" },
234 |           faculty: { full_name: "Dr. Nair" },
235 |         },
236 |       ],
237 |     };
238 |   }
239 |
240 |   const supabase = await createClient();
241 |   const { data, error } = await supabase
242 |     .from("assignments")
243 |     .select(`
244 |       id,
245 |       title,
246 |       description,
247 |       due_date,
248 |       max_marks,
249 |       file_url,
250 |       course:courses (code, title),
251 |       faculty:profiles (full_name)
252 |     `)
253 |     .order("due_date", { ascending: true });
254 |
255 |   if (error) return { success: false, message: error.message };
256 |   return { success: true, data: data || [] };
257 | }
258 |
259 | export async function createAssignmentAction(formData: unknown): Promise<ServerActionResponse> {
260 |   const parsed = assignmentSchema.safeParse(formData);
```

```
  if (!parsed.success) {
    return { success: false, errors: parsed.error.flatten().fieldErrors };
  }

  try {
    const { user, isMock } = await getAuthenticatedUserWithRoles(["faculty", "admin"]);
    if (isMock) {
      revalidatePath("/assignments");
      return { success: true, message: "Assignment created (Demo Mode)" };
    }

    const supabase = await createClient();
    const { data, error } = await supabase.from("assignments").insert({
      course_id: parsed.data.courseId,
      title: parsed.data.title,
      description: parsed.data.description,
      due_date: parsed.data.dueDate,
      max_marks: parsed.data.maxMarks,
      file_url: parsed.data.fileUrl || null,
      created_by: user.id,
    }).select().single();

    if (error) return { success: false, message: error.message };

    revalidatePath("/assignments");
    return { success: true, message: "Assignment published successfully!", data };
  } catch (err: any) {
    return { success: false, message: err.message };
  }
}

export async function submitAssignmentDeliverableAction(
  assignmentId: string,
  fileUrl: string,
  notes?: string
): Promise<ServerActionResponse> {
  try {
    const { user, isMock } = await getAuthenticatedUserWithRoles(["student"]);
    if (isMock) {
      revalidatePath("/assignments");
      return { success: true, message: "Assignment deliverable submitted (Demo Mode)" };
    }

    const supabase = await createClient();
    const { data, error } = await supabase
      .from("assignment_submissions")
      .upsert(
        {
          assignment_id: assignmentId,
          student_id: user.id,
          file_url: fileUrl,
          submission_notes: notes,
          status: "submitted",
          submitted_at: new Date().toISOString(),
        },
        { onConflict: "assignment_id,student_id" }
      )
      .select()
      .single();

    if (error) return { success: false, message: error.message };

    revalidatePath("/assignments");
    return { success: true, message: "Deliverable uploaded & receipt confirmed!", data };
  } catch (err: any) {
    return { success: false, message: err.message };
  }
}

export async function gradeSubmissionAction(formData: unknown): Promise<ServerActionResponse> {
  const parsed = submissionGradeSchema.safeParse(formData);
  if (!parsed.success) {
    return { success: false, errors: parsed.error.flatten().fieldErrors };
  }

  try {
    const { isMock } = await getAuthenticatedUserWithRoles(["faculty", "admin"]);
    if (isMock) {
      revalidatePath("/assignments");
      return { success: true, message: "Submission graded (Demo Mode)" };
    }

    const supabase = await createClient();
    const { error } = await supabase
      .from("assignment_submissions")
      .update({
        marks_received: parsed.data.marksReceived,
        feedback: parsed.data.feedback,
        status: "graded",
        graded_at: new Date().toISOString(),
      })
      .eq("id", parsed.data.submissionId);
```

```
353 |
354 |     if (error) return { success: false, message: error.message };
355 |
356 |     revalidatePath("/assignments");
357 |     return { success: true, message: "Grade committed to student record." };
358 |   } catch (err: any) {
359 |     return { success: false, message: err.message };
360 |   }
361 | }
362 |
363 | // ============================================================================
364 | // 4. Study Materials Actions
365 | // ============================================================================
366 |
367 | export async function getStudyMaterialsAction(courseId?: string): Promise<ServerActionResponse<any[]>> {
368 |   if (!isSupabaseConfigured()) {
369 |     return {
370 |       success: true,
371 |       data: [
372 |         {
373 |           id: "m-1",
374 |           title: "Unit 3: Relational Calculus & Query Optimization Slides",
375 |           description: "Comprehensive lecture slides for relational calculus and indexing.",
376 |           file_url: "/docs/dbms-unit-3.pdf",
377 |           file_size: "4.8 MB",
378 |           format: "PDF Presentation",
379 |           download_count: 142,
380 |           course: { code: "CS602", title: "Database Management Systems" },
381 |           faculty: { full_name: "Dr. Rajesh Varma" },
382 |         },
383 |       ],
384 |     };
385 |   }
386 |
387 |   const supabase = await createClient();
388 |   let query = supabase.from("study_materials").select(`
389 |     id,
390 |     title,
391 |     description,
392 |     file_url,
393 |     file_size,
394 |     format,
395 |     download_count,
396 |     created_at,
397 |     course:courses (code, title),
398 |     faculty:profiles (full_name)
399 |   `).order("created_at", { ascending: false });
400 |
401 |   if (courseId) {
402 |     query = query.eq("course_id", courseId);
403 |   }
404 |
405 |   const { data, error } = await query;
406 |   if (error) return { success: false, message: error.message };
407 |   return { success: true, data: data || [] };
408 | }
409 |
410 | export async function createStudyMaterialAction(formData: unknown): Promise<ServerActionResponse> {
411 |   const parsed = studyMaterialSchema.safeParse(formData);
412 |   if (!parsed.success) {
413 |     return { success: false, errors: parsed.error.flatten().fieldErrors };
414 |   }
415 |
416 |   try {
417 |     const { user, isMock } = await getAuthenticatedUserWithRoles(["faculty", "admin"]);
418 |     if (isMock) {
419 |       revalidatePath("/materials");
420 |       return { success: true, message: "Material uploaded (Demo Mode)" };
421 |     }
422 |
423 |     const supabase = await createClient();
424 |     const { data, error } = await supabase.from("study_materials").insert({
425 |       course_id: parsed.data.courseId,
426 |       title: parsed.data.title,
427 |       description: parsed.data.description,
428 |       file_url: parsed.data.fileUrl,
429 |       file_size: parsed.data.fileSize,
430 |       format: parsed.data.format,
431 |       uploaded_by: user.id,
432 |     }).select().single();
433 |
434 |     if (error) return { success: false, message: error.message };
435 |
436 |     revalidatePath("/materials");
437 |     return { success: true, message: "Study material published to course repo!", data };
438 |   } catch (err: any) {
439 |     return { success: false, message: err.message };
440 |   }
441 | }
442 |
443 | export async function deleteStudyMaterialAction(id: string): Promise<ServerActionResponse> {
444 |   try {
```

```
445 |     const { isMock } = await getAuthenticatedUserWithRoles(["faculty", "admin"]);
446 |     if (isMock) {
447 |       revalidatePath("/materials");
448 |       return { success: true, message: "Study material deleted (Demo Mode)" };
449 |     }
450 |
451 |     const supabase = await createClient();
452 |     const { error } = await supabase.from("study_materials").delete().eq("id", id);
453 |     if (error) return { success: false, message: error.message };
454 |
455 |     revalidatePath("/materials");
456 |     return { success: true, message: "Material removed." };
457 |   } catch (err: any) {
458 |     return { success: false, message: err.message };
459 |   }
460 | }
```

5. Frontend Design System & Key Component Implementations

The user interface follows a modern institutional SaaS design aesthetic featuring glassmorphism, responsive data grids, and accessible components. Below is the complete code for the main Navigation Bar and the Campus Statistics visualizer.

SOURCE FILE: src/components/layout/navbar.tsx **261 Lines**

```
"use client";

import * as React from "react";
import Link from "next/link";
import { usePathname } from "next/navigation";
import {
  GraduationCap,
  Bell,
  Search,
  PhoneCall,
  Menu,
  X,
  LogIn,
  UserPlus,
  Shield,
  Layers,
  Sparkles,
} from "lucide-react";
import { Button } from "@/components/ui/button";
import { Badge } from "@/components/ui/badge";
import { ThemeToggle } from "@/components/ui/theme-toggle";

export function Navbar() {
  const [mobileMenuOpen, setMobileMenuOpen] = React.useState(false);
  const pathname = usePathname();

  return (
    <header className="sticky top-0 z-40 w-full border-b border-zinc-200/80 bg-white/85 backdrop-blur-md dark:
  border-zinc-800/80 dark:bg-zinc-950/85 transition-colors">
      <div className="mx-auto flex h-16 max-w-7xl items-center justify-between px-4 sm:px-6 lg:px-8">
        {/* Brand */}
        <div className="flex items-center gap-6">
          <Link href="/" className="flex items-center gap-2.5 transition-transform hover:scale-[1.02]">
            <div className="flex h-10 w-10 items-center justify-center rounded-xl bg-gradient-to-tr from-blue-
  600 via-indigo-600 to-violet-600 shadow-md shadow-blue-500/20 text-white">
              <GraduationCap className="h-5 w-5" />
            </div>
            <div>
              <span className="text-base font-bold tracking-tight text-zinc-900 dark:text-zinc-50">
                SMART CAMPUS
              </span>
              <span className="block text-[10px] font-semibold tracking-wider text-blue-600 dark:text-blue-400
   uppercase">
                Enterprise Portal
              </span>
            </div>
          </Link>

          {/* Quick Links */}
          <nav className="hidden md:flex items-center gap-1 text-sm font-medium text-zinc-600 dark:text-zinc-3
  00">
            <Link
              href="/about"
              className={`rounded-lg px-3 py-1.5 transition-colors ${
                pathname === "/about"
                  ? "bg-zinc-100 text-zinc-900 dark:bg-zinc-800 dark:text-zinc-100 font-semibold"
                  : "hover:bg-zinc-100 dark:hover:bg-zinc-800/60"
              }`}
            >
              About
            </Link>
            <Link
              href="/features"
              className={`rounded-lg px-3 py-1.5 transition-colors ${
                pathname === "/features"
                  ? "bg-zinc-100 text-zinc-900 dark:bg-zinc-800 dark:text-zinc-100 font-semibold"
                  : "hover:bg-zinc-100 dark:hover:bg-zinc-800/60"
              }`}
            >
              Features
            </Link>
            <Link
              href="/contact"
              className={`rounded-lg px-3 py-1.5 transition-colors ${
                pathname === "/contact"
                  ? "bg-zinc-100 text-zinc-900 dark:bg-zinc-800 dark:text-zinc-100 font-semibold"
                  : "hover:bg-zinc-100 dark:hover:bg-zinc-800/60"
              }`}
            >
```

```
              Contact
            </Link>

            <span className="mx-1 h-4 w-px bg-zinc-200 dark:bg-zinc-800" />

            <Link
              href="/dashboard/student"
              className={`rounded-lg px-3 py-1.5 transition-colors ${
                pathname.startsWith("/dashboard/student")
                  ? "bg-blue-50 text-blue-600 dark:bg-blue-950/50 dark:text-blue-400 font-semibold"
                  : "hover:bg-zinc-100 dark:hover:bg-zinc-800/60"
              }`}
            >
              Student
            </Link>
            <Link
              href="/dashboard/faculty"
              className={`rounded-lg px-3 py-1.5 transition-colors ${
                pathname.startsWith("/dashboard/faculty")
                  ? "bg-amber-50 text-amber-600 dark:bg-amber-950/50 dark:text-amber-400 font-semibold"
                  : "hover:bg-zinc-100 dark:hover:bg-zinc-800/60"
              }`}
            >
              Faculty
            </Link>
            <Link
              href="/dashboard/admin"
              className={`rounded-lg px-3 py-1.5 transition-colors ${
                pathname.startsWith("/dashboard/admin")
                  ? "bg-rose-50 text-rose-600 dark:bg-rose-950/50 dark:text-rose-400 font-semibold"
                  : "hover:bg-zinc-100 dark:hover:bg-zinc-800/60"
              }`}
            >
              Admin
            </Link>
          </nav>
        </div>

        {/* Right Actions */}
        <div className="flex items-center gap-2 sm:gap-3">
          {/* Quick Search */}
          <Link
            href="/search"
            aria-label="Search Campus"
            className="hidden sm:flex items-center gap-2 rounded-xl border border-zinc-200 bg-zinc-50/80 px-3
py-1.5 text-xs text-zinc-500 hover:border-zinc-300 dark:border-zinc-800 dark:bg-zinc-900/80 dark:text-zinc-400
"
          >
            <Search className="h-3.5 w-3.5" />
            <span>Search modules...</span>
            <kbd className="rounded border border-zinc-200 bg-white px-1.5 py-0.5 text-[10px] text-zinc-400 da
rk:border-zinc-700 dark:bg-zinc-800">
              /
            </kbd>
          </Link>

          {/* Theme Toggle */}
          <ThemeToggle />

          {/* Notifications */}
          <Link
            href="/notifications"
            className="relative flex h-9 w-9 items-center justify-center rounded-xl border border-zinc-200 bg-
white/80 text-zinc-600 hover:bg-zinc-100 hover:text-zinc-900 dark:border-zinc-800 dark:bg-zinc-900/80 dark:tex
t-zinc-400 dark:hover:bg-zinc-800 dark:hover:text-zinc-100 transition-colors"
            aria-label="Notifications"
          >
            <Bell className="h-4 w-4" />
            <span className="absolute -top-1 -right-1 flex h-4 w-4 items-center justify-center rounded-full bg
-rose-500 text-[9px] font-bold text-white shadow-sm">
              3
            </span>
          </Link>

          {/* Emergency SOS Shortcut */}
          <Link
            href="/emergency"
            className="hidden lg:flex items-center gap-1.5 rounded-xl bg-rose-500/10 px-3 py-1.5 text-xs font-
semibold text-rose-600 hover:bg-rose-500/20 dark:bg-rose-500/20 dark:text-rose-400 transition-colors"
          >
            <PhoneCall className="h-3.5 w-3.5" />
            <span>SOS</span>
          </Link>

          {/* Auth Action Buttons */}
          <div className="hidden sm:flex items-center gap-2">
            <Link href="/login">
              <Button variant="ghost" size="sm" className="gap-1.5 text-xs font-semibold">
                <LogIn className="h-3.5 w-3.5" />
                Sign In
              </Button>
```

```
160 |             </Link>
161 |             <Link href="/register">
162 |               <Button size="sm" className="gap-1.5 text-xs font-semibold shadow-sm shadow-blue-500/20">
163 |                 <UserPlus className="h-3.5 w-3.5" />
164 |                 Register
165 |               </Button>
166 |             </Link>
167 |           </div>
168 |
169 |           {/* Mobile hamburger menu */}
170 |           <button
171 |             onClick={() => setMobileMenuOpen(!mobileMenuOpen)}
172 |             className="flex h-9 w-9 items-center justify-center rounded-xl border border-zinc-200 text-zinc-60
    |   0 md:hidden hover:bg-zinc-100 dark:border-zinc-800 dark:text-zinc-400 dark:hover:bg-zinc-800 transition-colors
    |   "
173 |             aria-label="Toggle navigation"
174 |           >
175 |             {mobileMenuOpen ? <X className="h-5 w-5" /> : <Menu className="h-5 w-5 text-zinc-700 dark:text-zin
    |   c-300" />}
176 |           </button>
177 |         </div>
178 |       </div>
179 |
180 |       {/* Mobile drawer */}
181 |       {mobileMenuOpen && (
182 |         <div className="border-b border-zinc-200 bg-white/95 px-4 pt-3 pb-6 md:hidden dark:border-zinc-800 dar
    |   k:bg-zinc-950/95 animate-fade-in backdrop-blur-lg">
183 |           <div className="space-y-1 pb-3 border-b border-zinc-100 dark:border-zinc-800 text-sm font-medium">
184 |             <p className="px-3 py-1 text-[11px] font-bold text-zinc-400 uppercase tracking-wider">
185 |               Navigation
186 |             </p>
187 |             <Link
188 |               href="/"
189 |               onClick={() => setMobileMenuOpen(false)}
190 |               className="block rounded-lg px-3 py-2 text-zinc-700 hover:bg-zinc-100 dark:text-zinc-300 dark:ho
    |   ver:bg-zinc-800"
191 |             >
192 |               Home
193 |             </Link>
194 |             <Link
195 |               href="/about"
196 |               onClick={() => setMobileMenuOpen(false)}
197 |               className="block rounded-lg px-3 py-2 text-zinc-700 hover:bg-zinc-100 dark:text-zinc-300 dark:ho
    |   ver:bg-zinc-800"
198 |             >
199 |               About Platform
200 |             </Link>
201 |             <Link
202 |               href="/features"
203 |               onClick={() => setMobileMenuOpen(false)}
204 |               className="block rounded-lg px-3 py-2 text-zinc-700 hover:bg-zinc-100 dark:text-zinc-300 dark:ho
    |   ver:bg-zinc-800"
205 |             >
206 |               Features & Modules
207 |             </Link>
208 |             <Link
209 |               href="/contact"
210 |               onClick={() => setMobileMenuOpen(false)}
211 |               className="block rounded-lg px-3 py-2 text-zinc-700 hover:bg-zinc-100 dark:text-zinc-300 dark:ho
    |   ver:bg-zinc-800"
212 |             >
213 |               Contact Support
214 |             </Link>
215 |           </div>
216 |
217 |           <div className="space-y-1 py-3 border-b border-zinc-100 dark:border-zinc-800 text-sm font-medium">
218 |             <p className="px-3 py-1 text-[11px] font-bold text-zinc-400 uppercase tracking-wider">
219 |               Portals
220 |             </p>
221 |             <Link
222 |               href="/dashboard/student"
223 |               onClick={() => setMobileMenuOpen(false)}
224 |               className="flex items-center justify-between rounded-lg px-3 py-2 text-blue-600 hover:bg-blue-50
    |    dark:text-blue-400 dark:hover:bg-blue-950/40"
225 |             >
226 |               <span>Student Dashboard</span>
227 |               <Badge variant="info">Portal</Badge>
228 |             </Link>
229 |             <Link
230 |               href="/dashboard/faculty"
231 |               onClick={() => setMobileMenuOpen(false)}
232 |               className="flex items-center justify-between rounded-lg px-3 py-2 text-amber-600 hover:bg-amber-
    |   50 dark:text-amber-400 dark:hover:bg-amber-950/40"
233 |             >
234 |               <span>Faculty Dashboard</span>
235 |               <Badge variant="warning">Portal</Badge>
236 |             </Link>
237 |             <Link
238 |               href="/dashboard/admin"
239 |               onClick={() => setMobileMenuOpen(false)}
240 |               className="flex items-center justify-between rounded-lg px-3 py-2 text-rose-600 hover:bg-rose-50
    |    dark:text-rose-400 dark:hover:bg-rose-950/40"
```

```
241 |             >
242 |               <span>Admin Command Center</span>
243 |               <Badge variant="destructive">Admin</Badge>
244 |             </Link>
245 |           </div>
246 |
247 |           <div className="pt-4 flex flex-col gap-2">
248 |             <Link href="/login" onClick={() => setMobileMenuOpen(false)}>
249 |               <Button variant="outline" className="w-full justify-center">
250 |                 Sign In
251 |               </Button>
252 |             </Link>
253 |             <Link href="/register" onClick={() => setMobileMenuOpen(false)}>
254 |               <Button className="w-full justify-center">Create Account</Button>
255 |             </Link>
256 |           </div>
257 |         </div>
258 |       )}
259 |     </header>
260 |   );
261 | }
```

SOURCE FILE: src/components/dashboard/attendance-chart.tsx **91 Lines**

```
  1 | "use client";
  2 |
  3 | import * as React from "react";
  4 | import {
  5 |   ResponsiveContainer,
  6 |   BarChart,
  7 |   Bar,
  8 |   XAxis,
  9 |   YAxis,
 10 |   Tooltip,
 11 |   CartesianGrid,
 12 | } from "recharts";
 13 |
 14 | interface AttendanceChartProps {
 15 |   data?: Array<{ subject: string; percentage: number }>;
 16 | }
 17 |
 18 | const defaultData = [
 19 |   { subject: "Data Structures", percentage: 88 },
 20 |   { subject: "Database Sys", percentage: 92 },
 21 |   { subject: "Algorithms", percentage: 81 },
 22 |   { subject: "Computer Networks", percentage: 76 },
 23 |   { subject: "Software Eng", percentage: 95 },
 24 | ];
 25 |
 26 | export function AttendanceChart({ data = defaultData }: AttendanceChartProps) {
 27 |   const [mounted, setMounted] = React.useState(false);
 28 |
 29 |   React.useEffect(() => {
 30 |     setMounted(true);
 31 |   }, []);
 32 |
 33 |   if (!mounted) {
 34 |     return (
 35 |       <div className="h-64 flex items-center justify-center text-xs text-zinc-400">
 36 |         Loading analytics...
 37 |       </div>
 38 |     );
 39 |   }
 40 |
 41 |   return (
 42 |     <div className="h-64 w-full">
 43 |       <ResponsiveContainer width="100%" height="100%">
 44 |         <BarChart data={data} margin={{ top: 10, right: 10, left: -20, bottom: 0 }}>
 45 |           <CartesianGrid strokeDasharray="3 3" opacity={0.15} />
 46 |           <XAxis
 47 |             dataKey="subject"
 48 |             tick={{ fontSize: 11 }}
 49 |             tickLine={false}
 50 |             stroke="#888888"
 51 |           />
 52 |           <YAxis
 53 |             domain={[0, 100]}
 54 |             tick={{ fontSize: 11 }}
 55 |             tickLine={false}
 56 |             stroke="#888888"
 57 |             unit="%"
 58 |           />
 59 |           <Tooltip
 60 |             content={({ active, payload }) => {
 61 |               if (active && payload && payload.length) {
 62 |                 return (
 63 |                   <div className="rounded-lg border border-zinc-200 bg-white p-2.5 shadow-md dark:border-zinc-
    |   800 dark:bg-zinc-900">
 64 |                     <p className="text-xs font-semibold text-zinc-900 dark:text-zinc-100">
 65 |                       {payload[0].payload.subject}
 66 |                     </p>
```

```
                    <p className="text-xs font-bold text-blue-600 dark:text-blue-400">
                      Attendance: {payload[0].value}%
                    </p>
                  </div>
                );
              }
              return null;
            }}
          />
          <Bar
            dataKey="percentage"
            fill="url(#blueGradient)"
            radius={[6, 6, 0, 0]}
          />
          <defs>
            <linearGradient id="blueGradient" x1="0" y1="0" x2="0" y2="1">
              <stop offset="0%" stopColor="#3b82f6" />
              <stop offset="100%" stopColor="#6366f1" />
            </linearGradient>
          </defs>
        </BarChart>
      </ResponsiveContainer>
    </div>
  );
}
```

9. Testing, Verification & Production Deployment

Quality assurance and production verification were executed across static type integrity, build compilation, and security checks.

Verification Aspect	Command / Tool	Expected Outcome	Status
Static Type Check	npx tsc --noEmit	Zero TypeScript compilation errors	PASSED
Code Standards & Linting	npm run lint	Full adherence to ESLint Next.js rules	PASSED
Production Build	npm run build	Optimized standalone bundle generation	PASSED
Git VCS Sync	git push origin main	Repository synchronized with GitHub remote	PASSED
Security Scan	Redaction Protocol	All API keys & tokens redacted with [REDACTED]	VERIFIED

Deployment Guidelines (Vercel & Netlify)

1. **Repository Connection:** Connect the GitHub repository (https://github.com/durgaprasadreddy557-ui/smartapp) to Vercel or Netlify.
2. **Environment Variable Provisioning:** Supply NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, and SUPABASE_SERVICE_ROLE_KEY in project settings.
3. **Continuous Integration:** Every push to branch main triggers an automated build running npm run build.

10. Future Scope & Conclusion

Key future enhancements envisioned for subsequent release phases include:

- **AI Attendance Insights:** Utilizing machine learning models to forecast student semester drop-out risks based on 30-day attendance trends.
- **Native Mobile Applications:** Developing React Native companion applications for iOS and Android with push notification delivery.
- **Biometric Hardware Integration:** Connecting RFID / BLE beacons in lecture halls to automate roll-call verification.
- **Online Fee Payment Gateway:** Incorporating Stripe / Razorpay for tuition fee and hostel billing.

Conclusion: Smart Campus provides higher education institutions with a modern, high-performance, and secure digital platform. By synthesizing real-time attendance calculation, academic coursework coordination, student grievances, and emergency services into a cohesive application, it sets a high benchmark for university institutional software.